UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA E INTERACCIÓN HUMANO COMPUTADORA

REPORTE DE PRÁCTICA Nº 05

NOMBRE COMPLETO: Vargas Luna José Ángel

Nº de Cuenta: 318252333

GRUPO DE LABORATORIO: 07

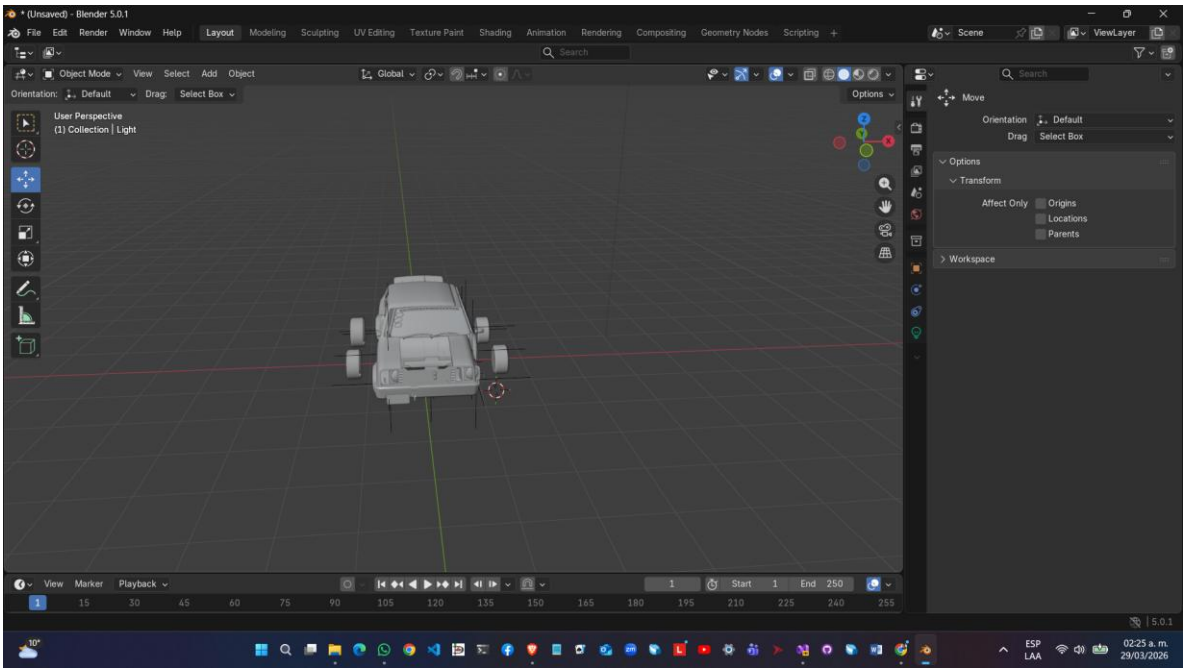
GRUPO DE TEORÍA: 05

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 29-03-2026

CALIFICACIÓN: _____

1.- Importar su modelo de coche propio dentro del escenario a una escala adecuada.



Modelo de mi auto en blender, donde cada parte que se trabaja, se separa y se empieza a manejar para tener su propio pivote en el centro del objeto

2.- Importar sus 4 llantas y acomodarlas jerárquicamente, agregar el mismo valor de rotación a las llantas para que al presionar puedan rotar hacia adelante y hacia atrás.

```
107
108        // --- LÓGICA DE LAS 8 TECLAS ---
109
110        if (mainWindow.getsKeys()[GLFW_KEY_W]) {
111            movCoche += 0.1f;
112            rotLlantas += 5.0f;
113        }
114
115        if (mainWindow.getsKeys()[GLFW_KEY_S]) {
116            movCoche -= 0.1f;
117            rotLlantas -= 5.0f;
118        }
```

Con las teclas W y S permite que el vehículo junto con las llantas avance hacia delante o hacia atrás.

3.- Importar el cofre del coche, acomodarlo jerárquicamente y agregar la rotación para poder abrir y cerrar.

```
        // --- MATRIZ MAESTRA (CARROCERÍA) ---
        glm::mat4 modelCarro = glm::mat4(1.0f);
        // Usamos movCoche aquí para que todo el grupo se mueva
        modelCarro = glm::translate(modelCarro, glm::vec3(0.0f, -1.0f, movCoche));
        glUniform3f(uniformColor, 0.2f, 0.2f, 0.2f);
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelCarro));
        Carroceria.RenderModel();

        // --- LLANTA DELANTERA DERECHA (DD) ---
        // Blender: X: 1.353, Y: -1.251, Z: 0.169
        glm::mat4 mDD = modelCarro;
        mDD = glm::translate(mDD, glm::vec3(1.353f, 0.169f,-1.251f)); // Mapeo: X, Z, Y
        mDD = glm::rotate(mDD, glm::radians(rotLlantas), glm::vec3(1, 0, 0)); // Prueba con (1,0,0) o (0,0,1)
        glUniform3f(uniformColor, 0.0f, 0.0f, 1.0f); // Azul
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(mDD));
        DD.RenderModel();

        // --- LLANTA DELANTERA IZQUIERDA (ID) ---
        // Blender: X: -1.226, Y: -1.278, Z: 0.147
        glm::mat4 mID = modelCarro;
        mID = glm::translate(mID, glm::vec3(-1.226f, 0.147f, -1.278f));
        mID = glm::rotate(mID, glm::radians(rotLlantas), glm::vec3(1, 0, 0));
        glUniform3f(uniformColor, 0.0f, 1.0f, 0.0f); // Verde
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(mID));
        ID.RenderModel();

        // --- LLANTA TRASERA DERECHA (TD) ---
        // Blender: X: 1.268, Y: 1.188, Z: 0.134
        glm::mat4 mTD = modelCarro;
        mTD = glm::translate(mTD, glm::vec3(1.268f, 0.134f,1.188f));
        mTD = glm::rotate(mTD, glm::radians(rotLlantas), glm::vec3(1, 0, 0));
        glUniform3f(uniformColor, 1.0f, 0.0f, 0.0f); // Rojo
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(mTD));
        TD.RenderModel();

```

```

        // --- LLANTA TRASERA IZQUIERDA (TI) ---
        // Blender: X: -1.395, Y: 1.189, Z: 0.140
        glm::mat4 mTI = modelCarro;
        mTI = glm::translate(mTI, glm::vec3(-1.395f, 0.140f,1.189f));
        mTI = glm::rotate(mTI, glm::radians(rotLlantas), glm::vec3(1, 0, 0));
        glUniform3f(uniformColor, 1.0f, 1.0f, 0.0f); // Amarillo
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(mTI));
        TI.RenderModel();

        // --- COFRE ---
        // Blender: X: 0.202, Y: -1.220, Z: 1.289
        glm::mat4 mCofre = modelCarro;
        mCofre = glm::translate(mCofre, glm::vec3(0.202f, 1.289f, 0.5f));
        mCofre = glm::rotate(mCofre, glm::radians(rotCofre), glm::vec3(1, 0, 0));
        glUniform3f(uniformColor, 1.0f, 0.0f, 1.0f); // Magenta
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(mCofre));
        Cofre.RenderModel();

        glUseProgram(0);
        mainWindow.swapBuffers();
    }
    return 0;
}
```

Jerarquía de mis objetos de mi carro.

4.- Agregar traslación con teclado para que pueda avanzar y retroceder de forma independiente

```
// --- LÓGICA DE LAS 8 TECLAS ---

if (mainWindow.getsKeys()[GLFW_KEY_W]) {
    movCoche += 0.1f;
    rotLlantas += 5.0f;
}

if (mainWindow.getsKeys()[GLFW_KEY_S]) {
    movCoche -= 0.1f;
    rotLlantas -= 5.0f;
}

if (mainWindow.getsKeys()[GLFW_KEY_O]) rotCofre += 2.0f;
if (mainWindow.getsKeys()[GLFW_KEY_P]) rotCofre -= 2.0f;
```

Conclusiones

Aunque esta práctica fue más pesada en cuestión del modelado e implementación de archivos .obj, estuvo interesante cómo ahora será más sencillo trabajar con los modelos de implementación de blender y ejecución en Visual Studio.

Referencias

Aurasoft. (2024). *STB_Image: Single-file public domain libraries for C/C++*. GitHub. https://github.com/nothings/stb/blob/master/stb_image.h

De Vries, J. (2023). *Learn OpenGL: Skybox*. LearnOpenGL. https://learnopengl.com/Advanced-OpenGL/Cubemaps

Khronos Group. (2024, 15 de enero). *Sampler (GLSL)*. OpenGL Wiki. https://wikis.khronos.org/opengl/Sampler_(GLSL)

Khronos Group. (2024, 2 de febrero). *Texture Filtering*. OpenGL Wiki. https://wikis.khronos.org/opengl/Sampler_(GLSL)